

NOUR MARNISSI

AI Pitch Deck Evaluator

A full-stack AI tool that reads pitch decks and returns structured investor-grade feedback — built from scratch with Node.js and Mistral AI.

Node.js

Mistral AI

Vanilla JS

REST API

JSON-mode LLM

What I built

The product

1. Upload a PDF pitch deck or paste slide text →
2. The system sends it to Mistral AI with a custom VC-persona prompt →
3. The model returns a structured JSON evaluation →
4. The frontend renders scores, strengths, weaknesses, and an investor verdict.



PDF + text input

FileReader API encodes the PDF to base64 client-side; Mistral reads it natively via `document_url`.



AI evaluation engine

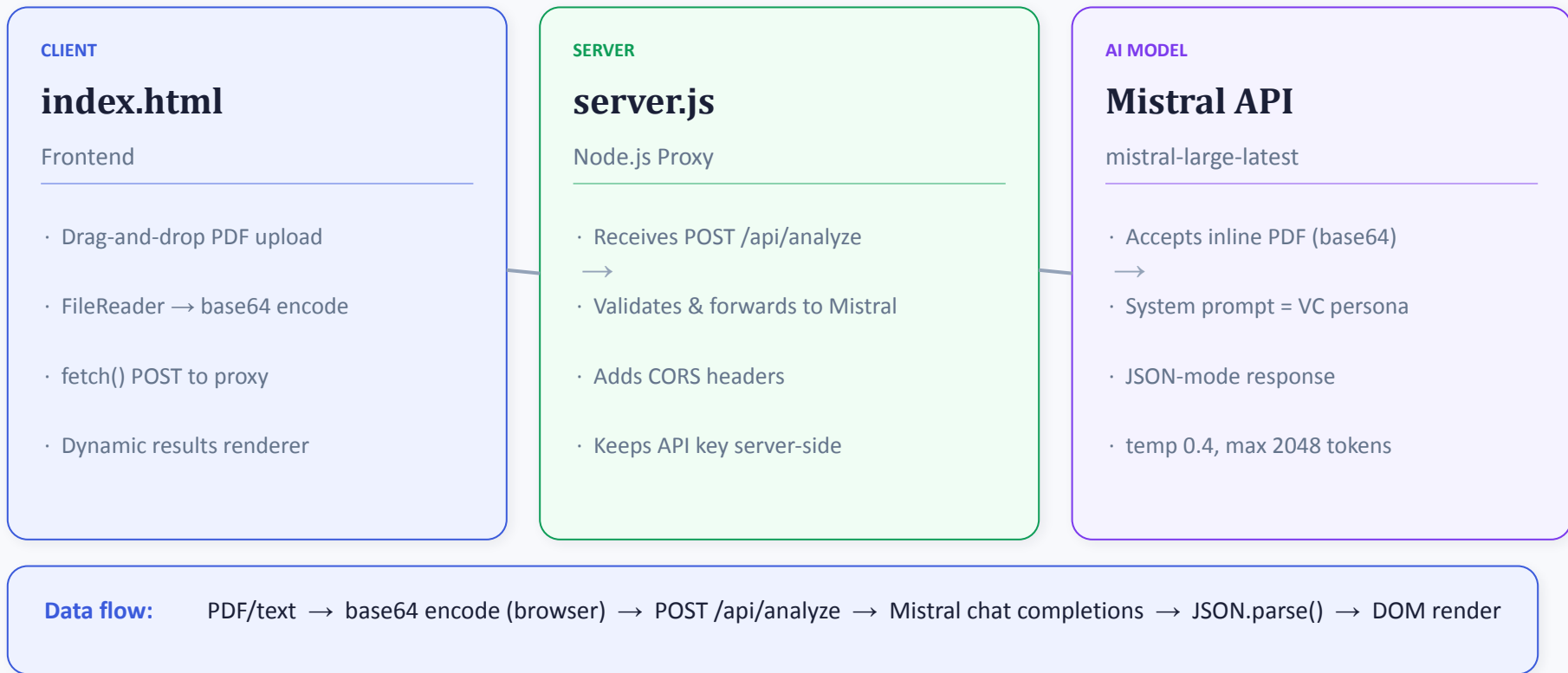
3 dimensions, scored 1–10, with strengths / weaknesses / investor insights per dimension.



Zero-dependency server

Pure Node.js stdlib (http + https). No npm install — any Node ≥ 14 just works.

How the system is designed



Key choices I made — and why

Model selection

Mistral Large over GPT-4

Native `document_url` support means I don't need a separate PDF-to-text parser. The base64 PDF is sent directly in the request body — one less dependency and fewer failure points.

Architecture

Node.js stdlib, no frameworks

Using only the built-in `http` / `https` modules kept the server to a single file with zero npm install. This minimises attack surface and deployment complexity — important for a proof-of-concept.

LLM engineering

JSON-mode (`response_format`)

Forcing structured output from the LLM means the frontend never has to parse free-form prose. The schema is defined in the system prompt; `JSON.parse()` handles the rest.

Security

Server-side API key proxy

The Mistral key never touches the browser. The client only calls `/api/analyze` on localhost. This is the correct pattern for any LLM-backed web app to prevent key leakage.

How I designed the evaluation prompt

Prompt design decisions

- **Persona engineering**
The model is given a specific identity: 'senior VC investor, 15+ years, hundreds of decks reviewed.' This grounds the scoring style rather than producing generic writing feedback.
- **Anchored scoring scale**
Explicit examples prevent score inflation: 8–10 = Airbnb/Dropbox tier. Without anchors, LLMs tend to cluster outputs at 7–8 for everything.
- **Forced JSON schema**
The schema is declared in the prompt. Each dimension must include score, summary, strengths[], weaknesses[], insights[]. This makes rendering deterministic.
- **Decision-framed verdict**
The final verdict asks 'would you take the next meeting?' — a real VC binary. This forces a concrete conclusion rather than hedged prose.

3 evaluation dimensions



Investor communication & clarity

Is the deck self-sufficient without the founder present?



Narrative & storytelling

Does it create urgency and make investors root for the team?



Problem-solution fit

Is the pain real? Is the solution differentiated?

What was hard — and what I learned



Challenge: Invalid header characters in API key

Learned: Zero-width spaces and BOM characters are invisible in editors but break HTTP headers. I added character-code validation at startup that catches them early and explains exactly which position is problematic.



Challenge: PDF encoding & Mistral's document API

Learned: Mistral's document_url accepts a data URI with base64 content. Getting the MIME type and encoding right took iteration — I learned how `FileReader.readAsDataURL()` works under the hood.



Challenge: LLM output is not always valid JSON

Learned: Even with JSON-mode enabled, the model sometimes wraps output in markdown fences. I strip ````json ... ```` before parsing. In production I'd add retry logic on parse failure.



Challenge: API key security in a client-server app

Learned: Early prototype exposed the key in the browser. I refactored to a Node proxy so the key lives only in `process.env` on the server — learning the correct pattern for LLM-backed web apps.

What I'd improve with more time

Current limitations



No authentication — `/api/analyze` is open to anyone on the same machine



10 MB PDF cap is enforced client-side only, not server-side



No retry on JSON parse failure — one bad model output shows an error



Stateless — results vanish on page refresh; no history or export



Single API call — very large decks (100+ slides) can hit token limits

If I had more time...



Add rate-limiting & auth — Session token or simple API key check on `/api/analyze`



Retry + fallback logic — 3 retries on JSON parse error; fallback to text extraction if PDF fails



Evaluation history — IndexedDB or lightweight SQLite to store past results and compare decks



Chunk large decks — Split long PDFs into sections, evaluate per-section, aggregate scores



Prompt A/B testing — Run multiple prompt variants, score calibration against human VC feedback

Thank you !

Key takeaways from this project:

- Built a real end-to-end AI product — no tutorials, just reading the Mistral API docs
 - Learned LLM prompt engineering hands-on: persona, JSON schema, scoring anchors
 - Practiced secure API architecture: server-side key, CORS, structured error handling
 - Identified real limitations and have a concrete plan to address each one
-